Task 2: Context-Aware NL2SPARQL with Interactive Linking

Helik Thacker, Wensheng Zhang, Julius Kaltwasser, Mauricio Kaupp Garcia, and Jeffry Cacho Aboukhalil

RWTH Aachen University, Templergraben 55, 52062 Aachen, Germany

{helik.thacker, wensheng.zhang, julius.kaltwasser, mauricio.kaupp, jeffry.cacho}@rwth-aachen.de

March 11, 2026

Abstract

Knowledge Graphs (KGs) like DBLP [1] provide vast repositories of structured data, yet their utility is often restricted by the requirement for SPARQL proficiency. While Large Language Models (LLMs) offer a promising bridge for natural language interfaces, they frequently struggle with entity ambiguity and schema hallucinations. This report presents a context-aware NL2SPARQL service designed to provide a robust abstraction layer over KGs.

Our system implements a modular pipeline that integrates a "user-in-the-loop" strategy, allowing for interactive entity linking and validation. To maintain KG-agnosticism and avoid the overhead of model retraining, we utilize dynamic "Context Packs" that provide runtime schema information and few-shot examples. Furthermore, we employ an autonomous tool-calling loop that enables the LLM to explore RDF ontologies and validate queries iteratively. Our results demonstrate that this combination of human-guided disambiguation and automated schema exploration improves query reliability and effectively lowers the technical barriers to semantic data access.

Our code can be found on GitLab.

1 Introduction

Knowledge Graphs store billions of RDF triples across research, business, and government domains, with repositories like DBLP containing more than 548 million RDF triples [2]. However, the accessibility of this data is limited. This is primarily due to the fact that users are expected to possess a high level of proficiency in SPARQL. Consequently, important information is not easily accessible.

To address this, we propose a Context-Aware NL2SPARQL service that translates natural language into SPARQL queries. In this report, we focus on a scholarly KG, DBLP. To achieve this, our system leverages LLMs. In order to overcome common LLM limitations, such as the hallucination of properties or incorrect entity identifiers, our pipeline incorporates an entity linking process and tools for the LLM to explore the ontology of the target RDF dataset.

2 Related Work

Developing a robust NL2SPARQL system involves two central challenges: mapping natural language mentions to entities in a KG (Entity Linking) and translating the intended meaning of the question into syntactically and semantically correct SPARQL queries (Query Generation).

Evolution of Entity Linking for DBLP Entity Linking in the scholarly domain is particularly challenging because author names are often ambiguous and schema definitions may evolve over time. **DBLPLink 1.0** [3] addressed this problem with a hybrid architecture that combines T5-based mention extraction with a

Siamese-network re-ranker built on KG embeddings such as TransE. While the system achieved competitive performance (F1-score ≈ 0.70), it depended on static embeddings, which introduced substantial maintenance overhead whenever the underlying graph changed.

To reduce this dependency on retraining, **DBLPLink 2.0** [4] moved to a zero-shot LLM-based approach. It scores candidates by using the log-probabilities of LLM-generated "yes" and "no" tokens over prompts built from linearized one-hop neighborhoods. This improves flexibility compared with embedding-based approaches. However, the system still depends on external search infrastructure such as Elasticsearch, and its prompts are tailored specifically to the DBLP dataset, which limits transferability to other KGs.

RAG and Schema Validation To improve portability beyond dataset-specific systems, recent work has explored augmenting LLMs with structured metadata. In particular, work on federated KGs [9] showed that Retrieval-Augmented Generation (RAG) with example queries and automatically generated ShEx schemas can improve SPARQL generation. Their approach uses a closed-loop validation mechanism that detects schema violations and prompts the LLM to repair the query. With this schema-aware repair step, they report an F1-score of 0.91 using GPT-4o. These results highlight the importance of a validation layer that constrains generation and supports automatic correction.

Generalization through Data Shapes For KGs that were not part of an LLM's training distribution, explicit structural guidance becomes even more important. Wardenga and Käfer [10] showed that providing data shapes such as ShEx or SHACL in the model context enables question answering over unseen public and private KGs. In their experiments, these structured constraints raised performance on unseen graphs from a baseline of zero to an F1-score of 0.28. This suggests that explicit schema-level guidance is essential for cross-KG generalization.

3 Proposed Solution - NL2SPARQL

Taken together, prior work showed that robust NL2SPARQL systems benefit from explicit schema grounding, validation, and methods that generalize beyond dataset-specific infrastructure. These observations motivate the design of our system. This section presents the proposed NL2SPARQL pipeline and its core design choices.

3.1 Context Pack

To guide the LLM throughout the pipeline, we use a schema-aware context pack. It is a structured, condensed representation of the KG's ontology: including classes, prefixes, predicates and relevant relations between classes. The goal is twofold: First, it enables the LLM to produce meaningful outputs by informing it with relevant context prior to the generation. Second, it enables the system to validate the generation and create a feedback loop for the LLM.

The context pack can additionally be extended with a few custom example queries that can help ground the LLM and stabilize the results. Especially in cases where the schema provided by the user is not fully aligned with the actual relations contained in the KG.

3.2 Mention Extraction

The first step of the pipeline begins with extracting mentions from the user's natural language query. A mention is defined as a contiguous span of the given input query that refers to a specific entity in the target KG. For example, in the input query "Who are the co-authors of Vaswani in Attention Is All You Need?", the spans *Vaswani* and *Attention Is All You Need* are the mentions referring to an author entity and a publication entity, respectively.

To perform this extraction, we utilize an LLM grounded by a schema-aware context pack. The system prompt (see Appendix A.1) is used along with the input query as the user prompt. Following the LLM's output, a validation layer ensures that the suggested types and predicates exist within the context pack and

are compatible. We specifically enforce that the identified label predicate has a range of `xsd:string` to facilitate candidate retrieval via SPARQL string matching.

Furthermore, in cases where a relation is semantically valid but would only be recoverable through reasoning, we adjust the extracted mention class to a directly queryable alternative. This avoids reliance on reasoning support at query time and reduces computational overhead on the DBLP endpoint.

Any mentions that fail to satisfy these schema constraints are discarded, ensuring that only valid, linkable entities proceed to the candidate generation stage.

3.3 Entity Linking

Extracted mentions are often lexically ambiguous. For instance, *Vaswani* may correspond to numerous distinct author nodes in the KG who share that name. To map a mention to its unique entity, we employ a two-stage entity linking process. First is the candidate generation, where we retrieve a set of potential matches from the SPARQL endpoint. This stage is followed by disambiguation, where the candidates are ranked to identify the specific node intended by the user's query.

Candidate Generation For each mention, we perform a multi-stage retrieval from the SPARQL endpoint to generate a candidate set. We first attempt a strict search for exact string matches between the mention text and the specified label predicate. If this yields no results, the system falls back to a relaxed search using token-based matching, which is computationally more expensive but ensures robustness against partial matches or minor variations. During this process, we distinguish between exact and partial matches to inform the subsequent ranking stage.

Disambiguation We employ a hybrid ranking strategy to select the most relevant entity from the candidate set. To provide sufficient textual context for ranking, we sample the one-hop neighborhood of each candidate node in parallel, constructing a "document" that describes the entity's graph context. The one-hop context is generated by retrieving the local neighborhood of an entity through three SPARQL queries: outgoing literal triples, outgoing IRI triples, and incoming IRI triples. For IRI neighbors, we resolve them to human-readable labels (rdfs:label) whenever possible. Finally, we use the BM25 algorithm to score these documents against the keywords extracted from the user's original natural-language query. To refine this statistical score, we apply several heuristics:

- **Exact Match Boost**: A significant constant weight is added to candidates whose labels exactly match the mention text.
- **Token Overlap**: A score proportional to the number of overlapping tokens between the mention and the candidate label is applied.
- **Graph Popularity**: In cases where textual context is sparse, we use the node degree (the number of incident edges) as a tie-breaker, favoring "denser" nodes which typically represent more prominent entities in the KG.

3.4 Query Generation

The Query Generation module transforms the user question and the previously linked entities into an executable SPARQL query. It combines structured prompt construction with a tool-calling loop for schema exploration and query generation.

Prompt Construction In the Prompt Construction step, the system builds the prompt (see Appendix A.2) sent to the LLM to prepare it for SPARQL query generation. The prompt is composed of the following elements:

- **System Guidelines:** A core set of instructions on how to generate the SPARQL query with tools for schema exploration and validation.

- **Contextual Examples:** A set of few-shot examples that demonstrate the correct SPARQL syntax and structure for the target domain.
- **Linked Entities:** The list of entities identified in the previous stages, including their specific IRIs and classes, providing the model with the exact identifiers needed for the query.
- **Original User Question:** The raw natural language input from the user, which serves as the primary source for the intended query logic.

Tool Calling Loop The agent is provided with tools (functions it can execute) to explore the schema. The following tools are available:

`list_classes` Lists all classes with IRIs and labels. The agent already receives one class IRI and label from linked mentions; this tool provides an overview and a starting point for exploration.

`describe_class` Given a class IRI, returns labels, comments, and super-classes. This is useful when a label alone is not sufficient for the agent to understand the meaning of the class, and it helps the agent understand the class hierarchy in the schema.

`describe_property` Given a property IRI, returns labels, comments, domain, range, super-properties, equivalent properties, and inverse properties. This follows the same idea as `describe_class`: the agent can call it when the label alone is not sufficient.

`get_incoming_properties` Given a class IRI, returns all properties for which that class is the range.

`get_outgoing_properties` Given a class IRI, returns all properties for which that class is the domain. The agent can use this tool together with `get_incoming_properties` to traverse the graph and find all required graph patterns for generating the correct query.

`run_sparql_query` Allows the agent to validate the query and run it on the endpoint. For invalid queries, it returns the corresponding error message; if validation is successful, it returns the query result. This allows the agent to verify that its query is valid and behaves as expected before returning it to the user.

We deliberately did not include a `list_properties` tool. For our test KG (DBLP), the number of available properties is very large, and returning them all in a single call would quickly exhaust the context window. Therefore, we opted for `get_outgoing_properties` and `get_incoming_properties`, which support more targeted schema exploration.

When designing the agent's toolbox, the goal was to remain token-efficient while still providing the LLM with all required information to quickly generate the correct query and debug it when the query is invalid or does not return the expected results.

Query Validation The validation stage first performs basic SPARQL syntax checks using `rdflib`. In particular, it relies on `parseQuery`, which also enforces that SPARQL Update operations (e.g., `DELETE`, `INSERT`, and `LOAD`) are not accepted.

Next, the validator checks namespace usage by extracting all prefixes declared in the query's `PREFIX` section and comparing them against a whitelist of valid namespaces stored in the `SchemaIndex`. Queries that use unknown or unauthorized namespaces are rejected.

The validator also verifies schema consistency: for all classes and properties under the schema base IRI, referenced classes and properties must exist in the schema index. To avoid false positives for concrete dataset instances returned by the entity-linking module, IRI checks are restricted to predicate positions and object positions that follow `rdf:type`.

Validation is executed both before a query is sent to the endpoint and when the user clicks the Validate button in the frontend.

Query Refinement Loop In the final UI, we provide a chat-based query-editing step that allows users to refine the generated SPARQL query with follow-up instructions, such as grouping results by year, sorting by a specific field, or including additional information. For each refinement turn, the system sends the original question, the current query, and the user's additional constraints back to the LLM agent, which then returns an updated query that preserves the original intent while incorporating the requested edits.

4 Evaluation

To evaluate our pipeline, we used a small subset of the DBLP benchmark containing 100 questions across all DBLP-QuAD [5] query types (e.g., `SINGLE_FACT`, `MULTI_FACT`), excluding `DISAMBIGUATION` queries. During preparation, we observed that the original dataset version was partially outdated. For example, venue information appears both as literals and as entities (streams), and several reference queries required adjustments to run on the current DBLP SPARQL endpoint.
Because our query-generation module depends on linked entities that include additional metadata (especially entity type), we enriched the evaluation input accordingly. After aligning the dataset and query assumptions with the current endpoint state, we re-executed all queries and stored the updated results in the evaluation dataset. The model used throughout the pipeline is `gpt-oss-120b`.

4.1 Mention Extraction

The mention extraction module was evaluated on the 100-query subset to assess its ability to identify and categorize entity surface forms (e.g., `Person`, `Publication`, `Stream`) from natural language input. The module achieved an F1-score of 0.99. We observed one failure case, in which the LLM also extracted the honorific titles (e.g., 'Dr.', 'Prof.') which led to a mismatch. In general, we found that the mentions are correctly extracted by the LLM guided by our context pack.

Our evaluation verified the extraction of the literal surface forms as they appeared in the question text. The model demonstrated high precision in identifying the exact tokens provided by the user, such as conference abbreviations (e.g., `ACSSC`) and name initials (e.g., `J. Mariéthoz`), without introducing hallucinations or premature label expansions. This high performance ensures that the downstream entity linking and query generation modules receive precise and structured input data.

4.2 Entity Linking

The entity linking module achieved an F1-score of 0.833 on the DBLP-QuAD subset. Our analysis reveals that the remaining discrepancies arise from three primary factors:

1. **Under-specified Ambiguity:** A significant portion of the linking "failures" is attributable to highly abbreviated person names in the benchmark (e.g., `Andrey Z.` or `Alberto R.`). In the DBLP database, these surface forms correspond to multiple distinct author profiles. Without additional context in the natural language query, the linking task is under-specified. In these scenarios, the system often linked to a valid literal match (e.g., `https://dblp.org/pid/186/1658` for `Andrey Z.`), which diverged from the specific entity intended by the gold standard (e.g., `https://dblp.org/pid/83/10230` for `Andrey Zhdanov`).

2. **Semantic and Structural Confusions:** The module occasionally failed to distinguish between closely related academic venues. For example, it linked `IEEE Wireless Communications` to the IRI for `IEEE Transactions on Wireless Communications` (stream `twc` instead of the expected `wc`). We also observed isolated placeholder errors, such as mapping the conference `MMAR` to a non-existent `gg` identifier.

3. **Substring Matching:** The system occasionally maps a mention to a candidate in which the mention appears only as a substring. For instance, the abbreviated name `B. Anda` was incorrectly mapped to `Kristine Claire B. Andaya` due to the high lexical overlap, rather than the intended literal match.

These results indicate that while the module is highly capable of grounding entities, its performance is primarily bounded by the ambiguity of abbreviated source text and occasional lexical biases in the ranking of candidate entities.

4.3 Query Generation

The evaluation compares executed outputs against gold results: for `ASK` queries, we compare boolean outcomes; for `SELECT` queries, we compare bindings. We require that the expected output column is present and that the number of solution mappings matches. At the same time, the matching logic is intentionally tolerant in cases where a semantically correct answer contains additional fields (e.g., returning both IRI and title for questions such as "Return all papers of author X").
Of the 100 queries, 86 matched completely, while 14 failed. We also observed no hallucinated entity IRIs or schema elements (properties/classes), as these are constrained by the validation layer.

5 Discussion

The evaluation shows that the proposed pipeline improves the reliability of LLM-based NL2SPARQL, particularly by reducing schema hallucinations and supporting more accurate entity grounding. At the same time, the results highlight several remaining challenges, including schema quality, entity ambiguity, endpoint limitations, and the function-calling reliability of the underlying LLM.

Interpretation of the Results The mention extraction results suggest that a schema-aware context pack is highly effective for constraining the model to valid entity types and surface forms. On the evaluated subset, the model extracted all mentions correctly, which indicates that prompt grounding with structured schema information can strongly stabilize this stage. However, this result should be interpreted with caution because the benchmark subset is small and partially adapted to the current endpoint state. The extraction score therefore demonstrates feasibility rather than definitive robustness across broader domains.

Entity linking proved to be substantially more challenging than mention extraction. The observed errors were concentrated in abbreviated person names, closely related venue names, and cases of strong substring overlap. This pattern is consistent with the structure of the DBLP benchmark itself: many surface forms are short and ambiguous, and in some cases the natural-language question does not provide enough information to uniquely identify the intended entity. As a result, part of the measured error reflects true ambiguity in the source input rather than a straightforward system failure. At the same time, the results also show that the current ranking strategy, based on BM25 and lightweight lexical heuristics, is still vulnerable and does not always distinguish between valid literal matches and the gold-standard entity intended by the benchmark.

The query generation results show a complementary pattern. Even when the generated query did not fully match the gold result, the validation layer successfully prevented hallucinated classes, properties, and entity IRIs. This is an important outcome because it demonstrates that explicit schema grounding and staged validation improve structural reliability, even when semantic interpretation is not fully correct. In other words, the system often fails in a controlled way: it may return an incomplete or incorrect query interpretation, but it is less likely to produce invalid or fabricated SPARQL.

Entity Linking Limitations The current entity-linking module is non-agentic and operates largely as a single-pass process. This keeps the pipeline simple, predictable, and computationally manageable, but it also introduces structural limitations. First, linking quality depends strongly on schema fidelity. The system assumes that the schema accurately reflects how the dataset is actually used. In practice, however, relevant predicates such as `rdfs:label` may be missing, underspecified, or inconsistently applied, which reduces candidate recall.

Second, SPARQL-based candidate generation is inherently limited by the search capabilities of the endpoint. Many SPARQL endpoints do not provide robust full-text retrieval, and regex-based fallback strategies can become prohibitively expensive. This makes candidate generation sensitive to lexical variation, abbreviations, and partial matches. Third, the current disambiguation strategy uses a lightweight one-hop textualization together with BM25, exact-match boosts, token overlap, and node-degree heuristics. While

this is a practical and transparent baseline, it reaches a performance ceiling on difficult ambiguities where more expressive neural re-rankers would likely perform better.

Finally, because the linker is non-agentic, it cannot iteratively inspect failures, reformulate retrieval strategies, or request clarification when confidence is low. As a consequence, early linking errors can propagate into later stages of the pipeline.

Query Generation Limitations The query generation stage depends heavily on the quality of both the linked entities and the schema representation. Although tool support and validation reduce hallucinations, they do not eliminate semantic ambiguity. Ambiguous natural-language questions can still lead the model toward an incorrect interpretation of the requested graph pattern, even when all referenced schema elements are valid.

A further limitation concerns the quality and representativeness of the schema itself. The system assumes that the schema is a useful proxy for the dataset, but this assumption does not always hold in practice. A concrete example is `dblp:authorOf`: although it appears in the schema as an `owl:inverseProperty`, it is not explicitly materialized in the DBLP triples, and the endpoint does not perform reasoning to infer it. Consequently, a structurally valid query may still return no results if it relies on schema relations that are not directly supported by the data. This highlights an important distinction between schema validity and dataset executability. In other words, a query can be correct with respect to the ontology and still fail operationally on a given endpoint.

In addition, the tool-calling loop depends on sufficiently reliable function calling by the LLM. Stronger models are better able to explore the schema, interpret tool outputs, and revise candidate queries. Weaker models may misuse tools, stop exploration too early, or fail to recover from execution errors. Thus, the effectiveness of the query generation module is partly coupled to the capabilities of the underlying model.

Dataset and Evaluation Constraints The evaluation itself introduces several constraints that should be considered when interpreting the results. First, the experiments were conducted on a relatively small subset of DBLP-QuAD, consisting of 100 questions. Second, the benchmark required adaptation because parts of the original dataset no longer aligned cleanly with the current DBLP endpoint. For example, venue information may appear both as literals and as entities, and some reference queries had to be adjusted to execute successfully. Third, the evaluation input was enriched with additional metadata required by the current pipeline, especially entity-type information for linked entities.

These adjustments were necessary to obtain a usable benchmark under current endpoint conditions, but they also mean that the reported scores are not directly comparable to results reported under older dataset snapshots or different execution settings. The current evaluation should therefore be understood as a realistic system-oriented assessment under present endpoint conditions rather than as a strict replication of prior benchmark numbers.

Broader Implications Taken together, the results support the central design claim of this report: robust NL2SPARQL benefits from combining explicit schema grounding, user-guided entity disambiguation, and validation-aware query generation. The main strength of the system is not that it solves all ambiguities perfectly, but that it reduces uncontrolled failures. Compared with one-shot LLM prompting, the pipeline shifts the problem toward more interpretable and more diagnosable failure modes, such as ambiguous abbreviations, schema-data mismatches, or endpoint limitations.

This is especially relevant for scholarly KGs such as DBLP, where ambiguity is common, schemas evolve, and endpoint behavior does not always match the ontology in a fully reasoning-aware way. In such settings, a modular architecture with validation and user control appears more practical than a purely end-to-end black-box approach. At the same time, the discussion also shows that true KG-agnostic robustness will require broader evaluation across multiple datasets and stronger mechanisms for handling ambiguity, retrieval failures, and schema-data divergence.

6 Future Work

The main next step is a true *end-to-end* evaluation of the agentic NL2SPARQL pipeline, not only of individual modules. Our current results are based on the non-agentic baseline for core stages, so a fair comparison should measure the full chain (mention extraction, linking, query generation, validation, and execution) under the same benchmark protocol. In addition to accuracy, this evaluation should report latency, token/tool-call cost, and failure modes, since practical usefulness depends on both correctness and efficiency.

A second research direction is improving agent robustness through better tools and better model-tool interaction. As frontier models continue to improve at function calling, performance will increasingly depend on tool quality: neural re-rankers for difficult disambiguation, semantic retrieval beyond lexical matching, pagination-aware exploration, and larger context windows for schema and evidence grounding. The goal is to move from brittle single-shot behavior to reliable iterative search-and-repair.

Third, prompt and interaction design remain open research questions. More guided clarification for ambiguous user intent, targeted follow-up questions, and prompt optimization across query types could improve end-to-end success without increasing user burden.

Finally, we need broader cross-KG validation. Although the architecture is designed to be KG-agnostic via context packs, experiments have focused mainly on DBLP. Systematic evaluation on Wikidata and Scholia is necessary to test transferability under schema and domain shift. Operational features such as admin/user management are relevant for deployment, but secondary to these research priorities.

7 Conclusion

In this report, we presented a context-aware NL2SPARQL pipeline for scholarly KGs, with DBLP as the main case study. The system combines schema-aware context packs, LLM-based mention extraction, interactive entity linking, tool-supported query generation, and multi-stage validation in order to translate natural-language questions into executable SPARQL queries more reliably.

Our results show that this modular design is effective at reducing two major weaknesses of LLM-based KG question answering systems: hallucinated schema elements and incorrect entity grounding. In particular, the evaluation indicates that schema-aware prompting strongly stabilizes mention extraction, while validation and tool support improve the structural correctness of generated queries. At the same time, the experiments also show that entity ambiguity, schema-data mismatches, and endpoint limitations remain important sources of error.

The main contribution of this work is therefore not a fully solved end-to-end NL2SPARQL system, but a practical architecture that makes failures more controlled, interpretable, and easier to diagnose. This is especially valuable in scholarly knowledge graphs such as DBLP, where ambiguity is common and schema structure does not always align perfectly with endpoint behavior.

Overall, the proposed approach demonstrates that robust NL2SPARQL benefits from combining explicit schema grounding, user-guided disambiguation, and validation-aware query generation. While broader evaluation across multiple knowledge graphs is still needed, the results suggest that this design is a promising step toward more reliable and accessible natural-language interfaces for semantic data.

References

[1] Ley, Michael. "The DBLP computer science bibliography: Evolution, research issues, perspectives." *International symposium on string processing and information retrieval*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002.

[2] DBLP Team: DBLP RDF statistics. `https://dblp.org/rdf/STATISTICS.txt` (last accessed: March 8, 2026).

[3] D. Banerjee, Arefa, R. Usbeck, and C. Biemann, "DBLPLink: An Entity Linker for the DBLP Scholarly Knowledge Graph," in *Proceedings of the ISWC 2023 Posters, Demos and Industry Tracks co-located with 22nd International Semantic Web Conference*, Athens, Greece, vol. 3632, CEUR-WS.org, 2023. [Online]. Available: `https://ceur-ws.org/Vol-3632/ISWC2023_paper_428.pdf`

[4] D. Banerjee, T. A. Taffa, and R. Usbeck, "DBLPLink 2.0 – An Entity Linker for the DBLP Scholarly Knowledge Graph," in *Proceedings of the ISWC 2025 Posters, Demos and Industry Tracks co-located with 24th International Semantic Web Conference*, Nara, Japan, vol. 4085, CEUR-WS.org, 2025. [Online]. Available: https://arxiv.org/abs/2507.22811

[5] Banerjee, P., et al.: DBLP-QuAD: A Question Answering Dataset over the DBLP Scholarly Knowledge Graph. In: Proceedings of the 22nd International Semantic Web Conference (ISWC 2023) (2023).

[6] W3C: SPARQL 1.1 Query Language. W3C Recommendation (21 March 2013). https://www.w3.org/TR/sparql11-query/

[7] Ollama Team: Ollama: Get up and running with LLMs locally. https://ollama.com/ (2024).

[8] RDFLib Team: RDFLib v7.0.0 documentation. https://rdflib.readthedocs.io/ (2024).

[9] D. Banerjee, P. Nayak, S. S. Shasidhar, and R. Usbeck, "LLM-based SPARQL Query Generation from Natural Language over Federated Knowledge Graphs," *arXiv preprint arXiv:2410.06062*, 2024. [Online]. Available: https://arxiv.org/abs/2410.06062

[10] J. G. Wardenga and T. Käfer, "Leveraging Data Shapes in Large Language Model Contexts for Question Answering on Public and Private Knowledge Graphs," in *Proceedings of the ISWC 2025 Posters, Demos and Industry Tracks co-located with 24th International Semantic Web Conference*, Nara, Japan, vol. 4094, CEUR-WS.org, 2025. [Online]. Available: https://ceur-ws.org/Vol-4094/paper1.pdf

A System Prompts

A.1 System Prompt for Mention Extraction

```
You are an entity mention extractor for a knowledge graph.

Goal
Extract entity mentions from the user query and select the correct STRING_LABEL predicate
to later retrieve matching entities from the knowledge graph.

Hard rules

- Only output spans that are contiguous substrings of the user text. NEVER invent text.
- Every mention text and every attr value MUST appear verbatim in the user query.
- Do not infer or complete missing information.
- Do not output duplicate (text, type) pairs.
- Return ONLY valid JSON.

How to choose `type`

- `type` MUST be exactly one of the Types listed in Schema Context.
- Output the TYPE CURIE exactly as shown (the left of the "|"), NOT the label text.

How to choose `label_pred` (CRITICAL)

- `label_pred` MUST be exactly one predicate CURIE listed under the chosen `type`.
- IMPORTANT: In Schema Context predicate rows have the shape:
    pred_curie | pred_label | rng:...
  You MUST output the pred_curie (left side). Ignore pred_label entirely.
- `label_pred` MUST have rng:lit(xsd:string).
- **Identity vs. Attribute**: Choose the predicate that stores the **Name, Label, or
  Identifier** of the entity itself.
- **CRITICAL**: Do NOT select a predicate just because its label appears in the user
query.
- If the user asks for "the *population* of **Paris**", the mention is "Paris" and the
`label_pred` should be a name/label predicate (e.g., `rdfs:label`), NOT the "population"
  predicate.
- The `label_pred` is used to *find* the entity by the `text` you extracted. The
attributes the user is asking *about* will be handled in a later step.
```

```
- Choose the predicate whose VALUES are most likely to contain the mention text verbatim.
- If multiple predicates are plausible, pick the best one.
- **CRITICAL**: You must extract the entity exactly as it appears in the source text,
character-for-character. Do not alter capitalization, accents, or punctuation. However,
if the entity is enclosed in quotation marks that act as delimiters in the query (e.g.,
'Paris' or "Nature"), do NOT include those surrounding quotes in the extraction.

Attrs

- attrs are optional metadata, NOT part of the mention text.
- Only use attribute keys that appear in Schema Context for that type.
- Values MUST be verbatim substrings of the user query.
- Do not emit empty strings.
- Year ranges:
  If a range appears (e.g., "2015-2020" or "between 2015 and 2020"),
  store it on ONE mention as "year_start":"2015","year_end":"2020".

Schema Context
{string_ctx}

Output JSON schema:
{{"mentions":[{{"text":str,"type":str,"label_pred":str,"attrs":{{str:str}}}}]}}
Return only JSON, no prose.
```

A.2 System Prompt for Query Generator

```
You are a SPARQL query generator.

You have access to schema exploration tools to discover:

- Classes
- Properties
- Domains and ranges

## Guidelines

- Do NOT assume class names or property names
- Do NOT hallucinate predicates
- Use the extracted mentions and their linked IRIs
- When doing SELECT queries, when returning a subject or object, always return the IRI in
addtion to results requested by the user.
- Only use string-based filtering as a last resort when no IRI is available for a mention.
In that case prefer `CONTAINS(LCASE(?var), LCASE("..."))` over exact literal equality.
- Use schema exploration tools before generating the SPARQL query
- ALWAYS run the query with `run_sparql_query` to validate the syntax and verify endpoint
execution behavior before returning it.
  - If `run_sparql_query` reports timeout, simplify the query and try again (fewer joins/
  filters, add stricter constraints, add reasonable LIMIT).
  - If `run_sparql_query` reports syntax/forbidden/endpoint error, revise the query before
   returning.
- Always include a PREFIX declaration section at the top of the query using standard
abbreviations (e.g., rdfs:, xsd:, dblp:) to ensure the SELECT clause remains concise
and readable.
- Return ONLY the SPARQL query
- Do NOT add comments or explanations

## Examples

{examples}
```